

PROJECT REPORT

Automated Phishing Email Detection Using VirusTotal and Splunk

1. Introduction

Phishing attacks remain one of the most common entry points for cyber threats. In this project, we designed and implemented an automated system capable of detecting phishing emails by integrating email processing, VirusTotal, and Splunk Enterprise.

The idea was simple at first glance. But once we got into it, it became clear that automation, APIs, and system integration rarely behave perfectly on the first try.

This project demonstrates how phishing emails can be:

- Simulated
- Analyzed automatically
- Forwarded into a monitoring system
- Logged for security awareness and response

2. Objectives

The main objectives of this project were:

- To simulate phishing email attacks between controlled accounts
- To integrate VirusTotal API for email analysis
- To configure Splunk for real-time event monitoring
- To automate detection and alerting workflow
- To understand real-world challenges in cybersecurity automation

3. Tools and Technologies Used

- Python (for scripting and automation)
- VirusTotal API
- Splunk Enterprise
- Gmail accounts (for sender and receiver simulation)
- HTTP Event Collector (HEC) in Splunk
- Python libraries such as requests and .env

4. System Architecture Overview

The system was designed in three main stages:

1. Email Simulation

- a. One Gmail account acts as attacker
- b. Another Gmail account acts as victim

2. Email Analysis

- a. Emails are processed and sent to VirusTotal via API
- b. URLs, domains, and metadata are checked

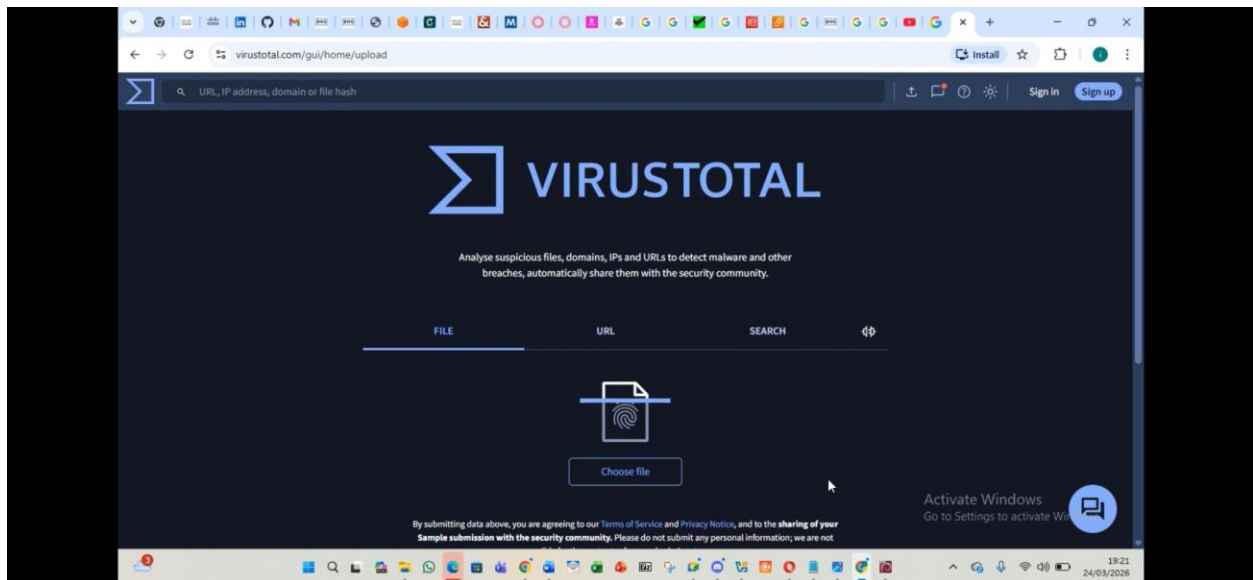
3. Event Logging

- a. Results are forwarded to Splunk
- b. Splunk stores and visualizes the events

5. Implementation Process

5.1 Setting Up VirusTotal API

We created a VirusTotal account and generated an API key from the dashboard.



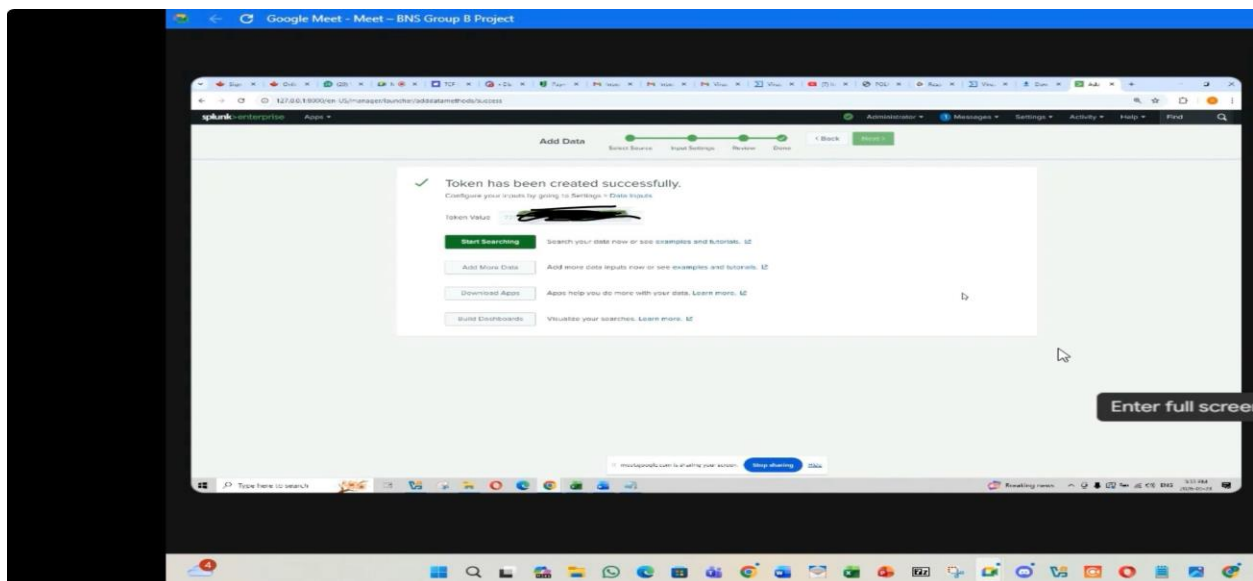
VirusTotal dashboard showing generated API key used for malware and phishing analysis.

5.2 Configuring Splunk Enterprise

Splunk was installed locally and accessed via:

<http://127.0.0.1:8000>

We enabled the HTTP Event Collector and created a token.



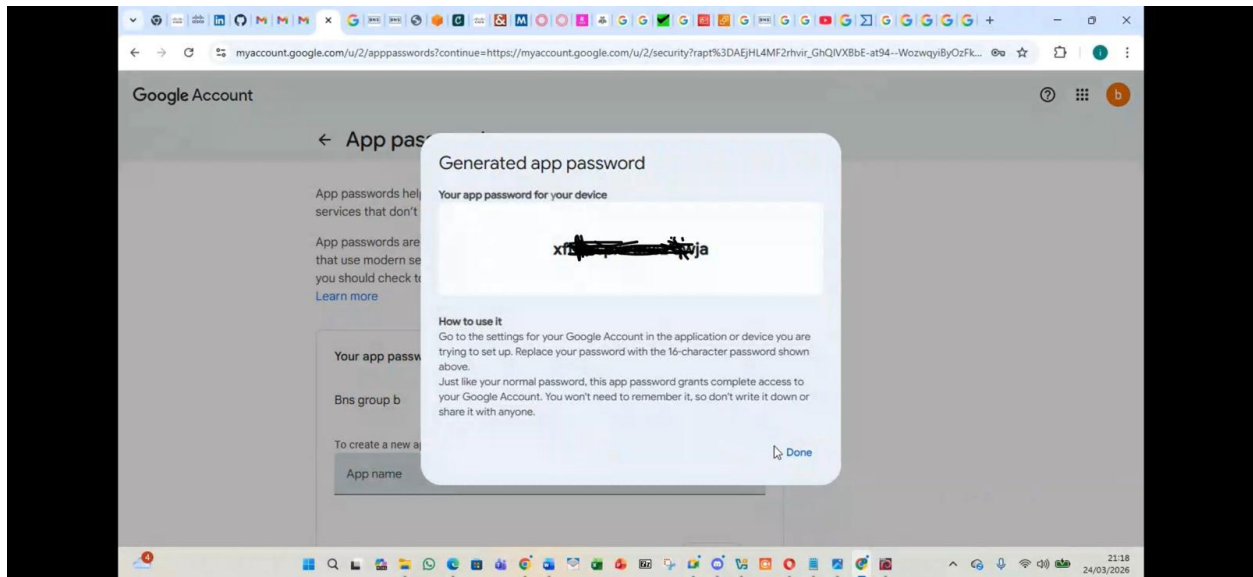
Splunk HTTP Event Collector configuration showing generated token for event ingestion.

5.3 Creating Email Accounts

Two Gmail accounts were created:

- Sender email (attacker simulation)
- Receiver email (target simulation)

We generated **App Passwords** by enabling 2FA.



Gmail security settings showing App Password generation for secure email automation.

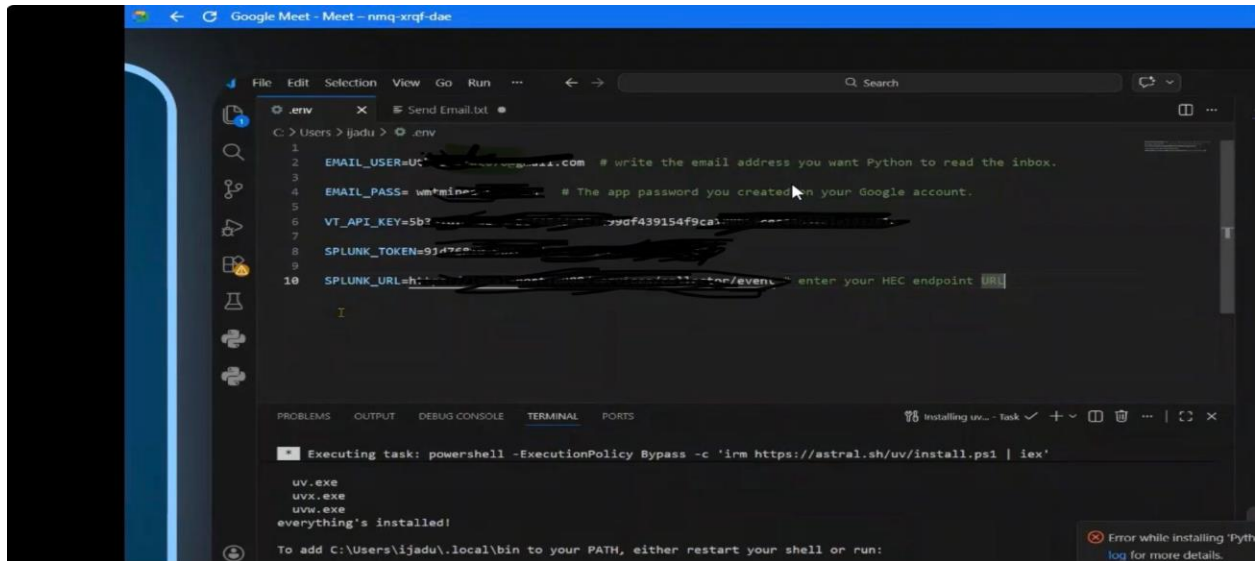
5.4 Writing Python Scripts

We worked with three main scripts:

a. Environment Configuration (.env)

Contained:

- Email credentials
- VirusTotal API key
- Splunk token
- Splunk URL

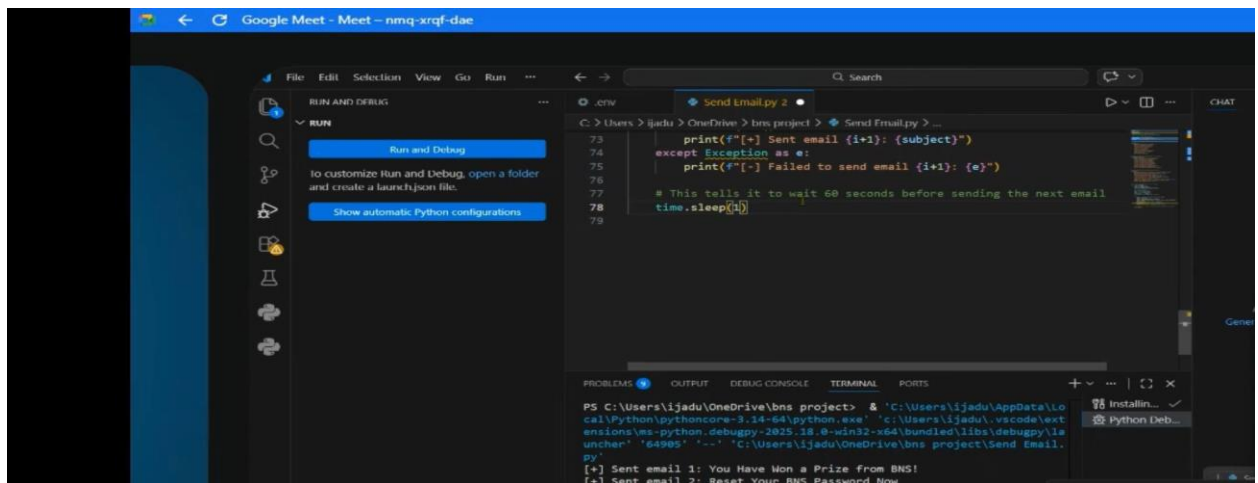


b. Email Sender Script

This script simulated phishing emails being sent repeatedly.

We adjusted timing:

- Changed delay from 60 seconds to 1 second for testing



Python script execution sending multiple simulated phishing emails to target inbox.

c. Email Analysis and Splunk Integration Script

This script:

- Reads incoming emails
- Sends data to VirusTotal

- Forwards results to Splunk

6. Challenges Encountered

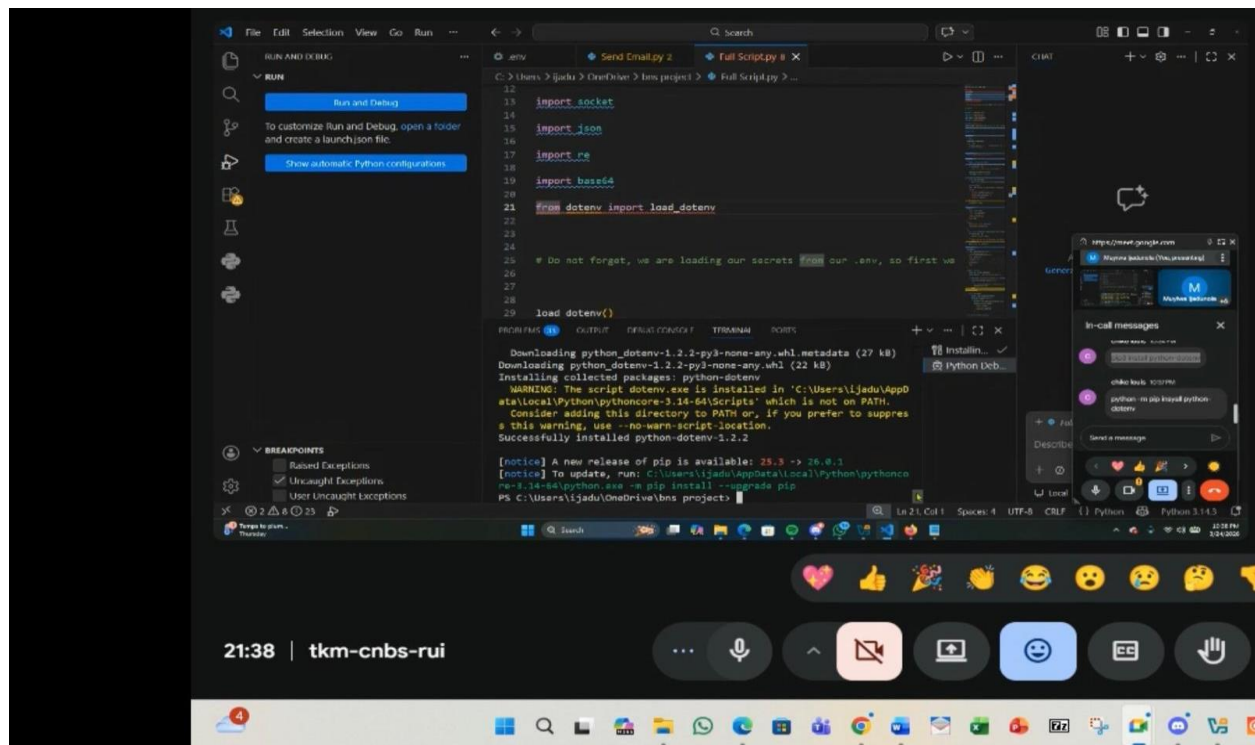
Honestly, this is where most of the real learning happened.

6.1 Python Environment Issues

- Missing modules like requests and dotenv
- Fixed using:

`python -m pip install requests`

`python -m pip install python-dotenv`



6.2 Email Delivery Errors

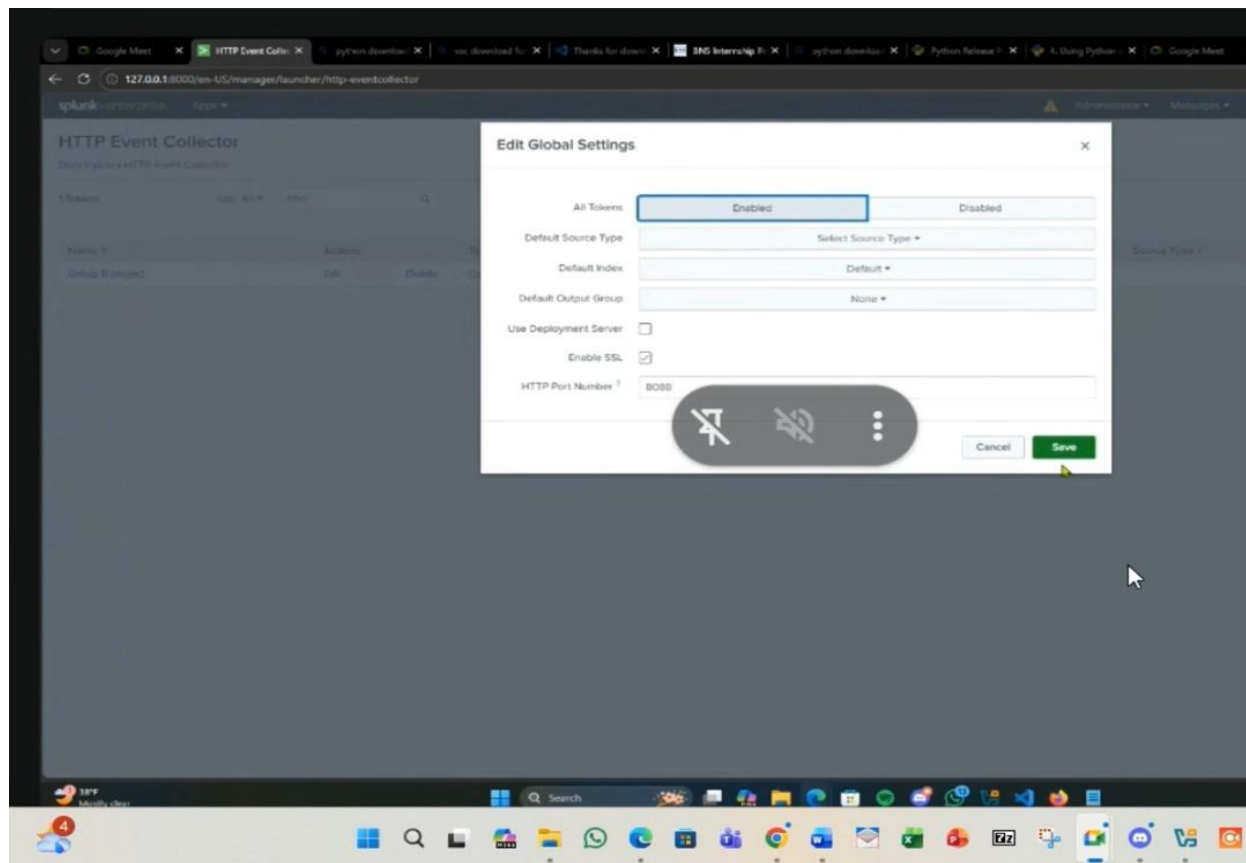
- Incorrect email address caused bounce-back messages
- Fixed by verifying correct recipient email

6.3 App Password Issues

- Gmail required 2FA before generating app passwords

6.4 Splunk Connection Errors

- HTTP Event Collector was initially disabled
- Fixed by enabling it in global settings



6.5 API Limitations (VirusTotal)

This one was subtle.

- Free API only allows limited requests

- Some emails were not flagged as threats
- Error like “*cannot canonicalize URL*” appeared

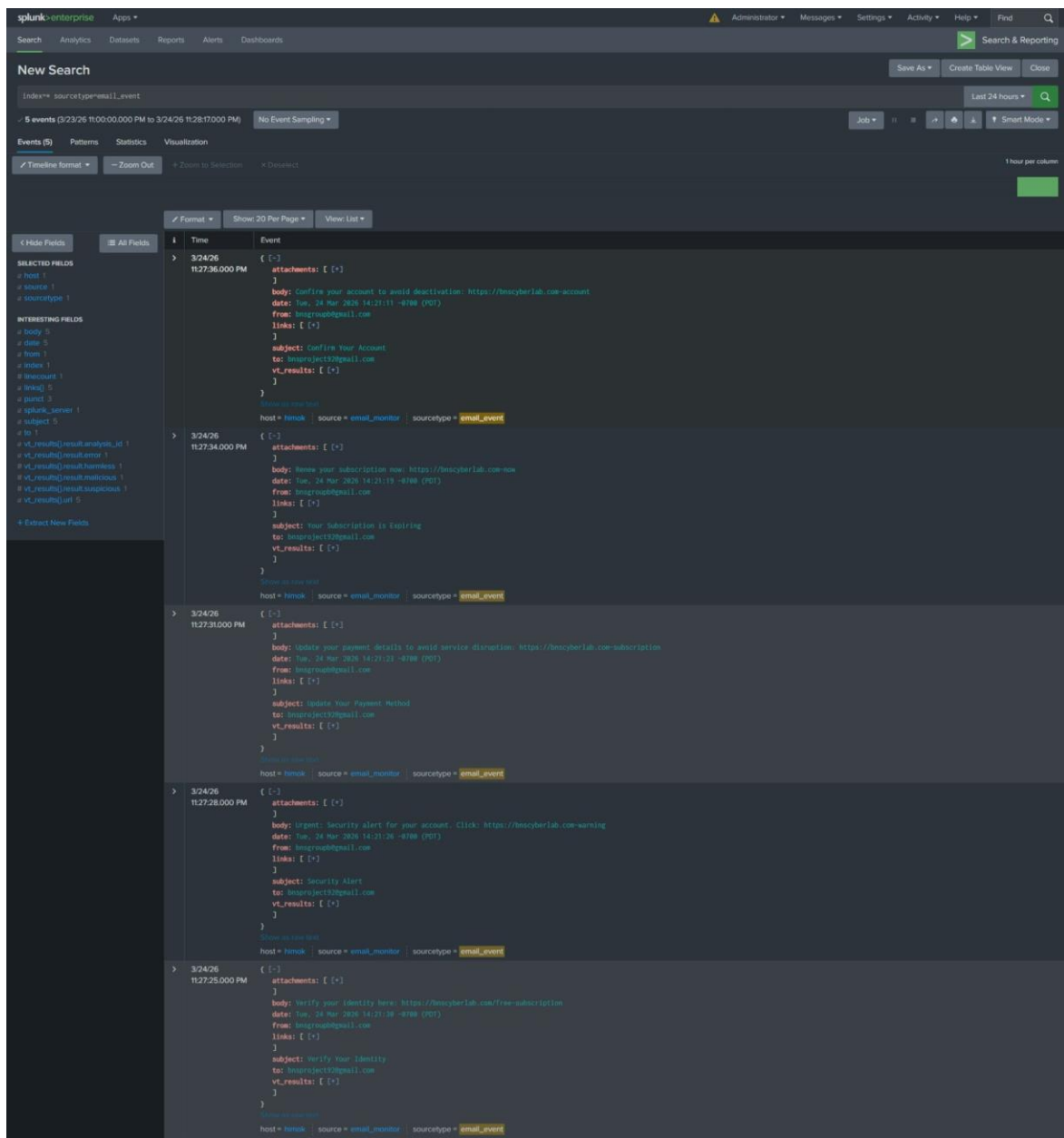
Conclusion:

- Not all emails were detected because they were not malicious
- Also limited by free API constraints

7. Results

After proper configuration:

- Emails were successfully sent from attacker to victim
- Emails were processed and analyzed
- Events were forwarded to Splunk
- Splunk displayed logs and metadata



Splunk dashboard displaying indexed email events including sender, subject, and analysis results.

8. Key Observations

- Automation works, but only after careful debugging
- API limitations can affect detection accuracy

- System integration requires consistency (same machine, same network)
- Small configuration issues can break the entire pipeline

9. Conclusion

This project successfully demonstrated a working prototype of an automated phishing email detection system.

Even though some limitations existed, especially with the free version of VirusTotal, the overall system achieved its purpose:

- Simulating attacks
- Processing email data
- Logging events into a SIEM system

It also highlighted something important...

Real cybersecurity work is rarely smooth. You fix things as you go.